

Arca

Manual técnico

Instalação, uso, operação e extensão do banco de dados Arca

Servidor em C++20 com LSM-Tree, protocolo compatível com o Redis, tabelas com SQL, API REST em Go e painel web. Este manual cobre do primeiro uso à operação em produção.



Versão 1.0.0 · LSM-Tree-Database-Server · Copyright (©) 2026 Erik Pimenta · Licença MIT

Sumário

1 Visão geral

2 Instalação

3 Primeiros passos

4 O servidor e os comandos

5 SQL

6 O painel

7 API REST

8 Configuração

9 Operação

10 Estendendo a Arca

11 Solução de problemas

1 Visão geral

A Arca é um banco de dados que guarda dois tipos de dado no mesmo motor de armazenamento: pares chave-valor, no estilo do Redis, e tabelas com colunas tipadas consultadas por SQL. Tudo é gravado em arquivos binários próprios organizados como uma **LSM-Tree** (Log-Structured Merge-Tree), a mesma família de estrutura usada por LevelDB, RocksDB e Cassandra.

O projeto é dividido em três programas que podem rodar juntos ou separados:

Programa	Linguagem	Função
`arca-server`	C++20	Motor LSM, rede TCP assíncrona, comandos, SQL, expiração de chaves
`arca-gateway`	Go 1.24	API REST, sessões, papéis, auditoria, métricas em tempo real e entrega do painel
Painel (`web/`)	React + TypeScript	Interface para ver níveis e SSTables, navegar em chaves e tabelas e rodar SQL
`arca-cli` , `arca-bench` , `arca-inspect`	C++20	Console, teste de carga e inspeção dos arquivos binários

Por que uma LSM-Tree

Bancos baseados em B-Tree atualizam páginas no lugar, o que gera escrita aleatória no disco. A LSM-Tree transforma toda escrita em escrita sequencial: primeiro no log, depois em uma estrutura em memória e, quando ela enche, em um arquivo ordenado e imutável. Arquivos antigos são mesclados em segundo plano. O resultado é alta vazão de escrita, recuperação simples após queda e leituras rápidas com a ajuda de filtros de Bloom e cache de blocos.

Garantias

- **Durabilidade:** com `appendfsync always` , uma escrita confirmada sobrevive a queda de energia. Com `everysec` (padrão), perde-se no máximo cerca de um segundo.
- **Atomicidade:** cada comando e cada instrução SQL viram um único lote gravado de uma vez no log.
- **Isolamento:** leituras usam um número de sequência e nunca enxergam lotes pela metade. `MULTI/EXEC` executa os comandos enfileirados sem intercalar outros clientes.
- **Integridade:** todo registro do log e todo bloco de SSTable têm checksum CRC32C.

2 Instalação

Com Docker (recomendado)

É o caminho mais simples em qualquer sistema. Instale o Docker Desktop (Windows e macOS) ou Docker Engine com Compose (Linux).

1. Baixe o repositório e abra a pasta `LSM-Tree-Database-Server`.
2. **Windows:** dê dois cliques em `iniciar.cmd`. **Linux ou macOS:** rode `./iniciar.sh`.
3. Na primeira vez o script cria o arquivo `.env` com uma senha aleatória para o servidor, compila as três imagens e sobe os contêineres.
4. Quando o painel responder, o navegador abre em **http://localhost:8080**.

```
cp .env.example .env              # sem o script: troque ARCA_PASSWORD
docker compose up -d --build
docker compose logs -f arca       # acompanha o servidor
docker compose down               # para tudo (os dados ficam no volume "dados")
```

Compilando o servidor

O núcleo usa só a biblioteca padrão do C++. Requisitos: GCC 11+, Clang 14+ ou Visual Studio 2022, e CMake 3.20+.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel
./build/arca-tests                 # 19 testes do motor, do SQL e do servidor
./build/arca-server config/arca.conf
```

No Windows, use `cmake --build build --config Release`; os executáveis ficam em `build\Release\`. A rede usa epoll no Linux, poll no macOS e WSAPoll no Windows, escolhidos na compilação.

Gateway e painel sem Docker

```
cd web && npm ci && npm run build            # gera web/dist
cd ../gateway
ARCA_ADDR=127.0.0.1:7379 ARCA_PASSWORD=senha go run ./cmd/arca-gateway
```

O gateway procura o painel em `../web/dist`; mude com `ARCA_WEB_DIR`. Para desenvolver o painel com recarga automática, rode `npm run dev` e abra http://localhost:5173 (as chamadas para `/api` vão para a porta 8080).

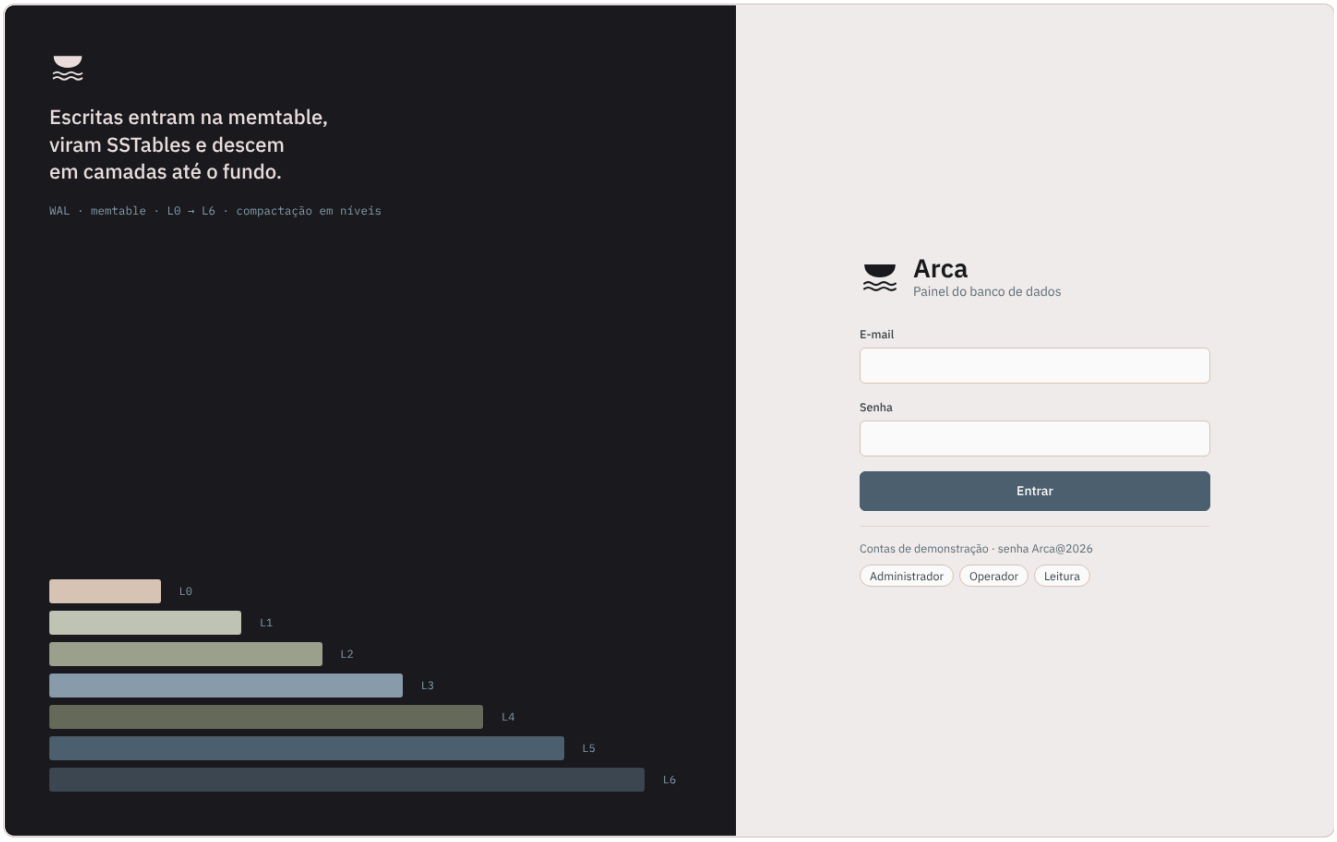
3 Primeiros passos

Entrando no painel

Na primeira subida o gateway cria três contas de demonstração e dados de exemplo. Todas usam a senha `Arca@2026`.

Conta	Papel	Permissões
`admin@arca.dev`	Administrador	Tudo, inclusive usuários, configuração, auditoria e console de comandos
`operador@arca.dev`	Operador	Lê e grava chaves, linhas e SQL; grava a memtable, compacta e cria checkpoints
`leitura@arca.dev`	Leitura	Só consulta

> As contas existem só para teste. Em uso real, suba com `ARCA_SEED=false` num volume vazio, crie um administrador e troque as senhas.



Tela de entrada do painel

Usando o redis-cli

```
redis-cli -p 7379 -a "$ARCA_PASSWORD"
127.0.0.1:7379> SET sessao:42 ativo EX 3600
OK
127.0.0.1:7379> TTL sessao:42
(integer) 3600
127.0.0.1:7379> INCRBY contador:visitas 5
(integer) 5
```

Usando o arca-cli

O `arca-cli` abre um console que aceita comandos e SQL. Linhas que começam com SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, ALTER, SHOW, DESCRIBE ou EXPLAIN são enviadas como SQL e o resultado aparece em tabela.

```
$ arca-cli -a senha
127.0.0.1:7379> SELECT categoria, COUNT(*) AS itens FROM produtos GROUP BY categoria
+------------+-------+
| categoria  | itens |
+------------+-------+
| Papelaria  | 9     |
| Eletrônicos| 8     |
+------------+-------+
(2 linhas)
```

4 O servidor e os comandos

Como o servidor atende

O `arca-server` abre um socket de escuta e cria `io-threads` laços de eventos. O primeiro laço aceita conexões e as distribui em rodízio. Cada laço lê os dados dos clientes sem bloquear, decodifica o protocolo RESP2, executa o comando e devolve a resposta. Vários comandos podem chegar de uma vez (pipelining) e são respondidos na ordem.

Comandos que podem demorar (SQL, KEYS, DBSIZE, FLUSHDB, compactação, checkpoint) vão para um pool de `worker-threads`. Enquanto isso a conexão fica pausada para preservar a ordem das respostas, mas o laço continua atendendo os outros clientes.

Grupo	Comandos
Conexão	PING · ECHO · AUTH · HELLO · SELECT 0 · QUIT · CLIENT ID/SETNAME/GETNAME/LIST
Strings	GET · SET (EX PX EXAT PXAT NX XX GET KEEPTTL) · SETNX · SETEX · PSETEX · GETSET · GETDEL · MGET · MSET · APPEND · STRLEN
Contadores	INCR · DECR · INCRBY · DECRBY · INCRBYFLOAT
Chaves	DEL · UNLINK · EXISTS · TYPE · RENAME · RENAMENX · EXPIRE · PEXPIRE · EXPIREAT · PEXPIREAT · TTL · PTTL · PERSIST · SCAN · KEYS · DBSIZE · FLUSHDB
Transações	MULTI · EXEC · DISCARD
SQL	SQL consulta · SQL.JSON consulta · SQL.TABLES
Servidor	INFO [seção] · CONFIG GET/SET/RESETSTAT · SLOWLOG GET/LEN/RESET · SAVE · BGSAVE · LASTSAVE · TIME · SHUTDOWN [NOSAVE]
Arca	ARCA.STATS · ARCA.FLUSH · ARCA.COMPACT · ARCA.CHECKPOINT [nome] · ARCA.SNAPSHOT BEGIN/END · ARCA.SCANJSON cursor padrão quantidade

Expiração de chaves

Uma chave com TTL guarda o instante de expiração junto do valor e ganha uma entrada num índice ordenado por tempo. Leituras já tratam a chave vencida como inexistente; uma thread percorre esse índice a cada 100 ms e apaga as chaves vencidas de fato, liberando espaço na próxima compactação.

Transações

Depois de `MULTI`, os comandos são validados e enfileirados. `EXEC` roda todos em sequência com exclusividade: nenhum outro cliente executa nada no meio. Se algum comando enfileirado tiver erro de sintaxe, `EXEC` responde `EXECABORT` e nada é aplicado.

Snapshot por conexão

`ARCA.SNAPSHOT BEGIN` fixa um retrato do banco para a conexão: até `ARCA.SNAPSHOT END`, GET, SCAN e SELECT enxergam os dados como estavam naquele instante, mesmo com escritas e compactações

acontecendo.

5 SQL

As tabelas vivem na mesma LSM-Tree das chaves. Cada linha é um par chave-valor cuja chave é o identificador da tabela mais a chave primária codificada de forma que a ordem dos bytes respeite a ordem dos valores. Isso permite busca direta pela chave primária e varreduras ordenadas.

Tipo	Aceita	Observação
INT	INT, INTEGER, BIGINT, SMALLINT, TIMESTAMP	Inteiro de 64 bits
REAL	REAL, FLOAT, DOUBLE, NUMERIC, DECIMAL	Ponto flutuante de 64 bits; aceita literal inteiro
TEXT	TEXT, VARCHAR(n), STRING, CHAR	UTF-8, sem limite de tamanho
BOOL	BOOL, BOOLEAN	TRUE ou FALSE

Instruções

```
CREATE TABLE [IF NOT EXISTS] t (id INT PRIMARY KEY AUTOINCREMENT, nome TEXT NOT NULL, ativo
BOOL DEFAULT TRUE)
DROP TABLE [IF EXISTS] t
ALTER TABLE t ADD COLUMN email TEXT [DEFAULT 'x']
CREATE INDEX [IF NOT EXISTS] idx ON t (coluna)        DROP INDEX [IF EXISTS] idx
INSERT INTO t [(colunas)] VALUES (...), (...)
SELECT * | expressão [AS apelido], COUNT(*), SUM(c), AVG(c), MIN(c), MAX(c) FROM t
       [WHERE expr] [GROUP BY c, ...] [ORDER BY c [ASC|DESC], ...] [LIMIT n [OFFSET m]]
UPDATE t SET c = expr, ... [WHERE expr]
DELETE FROM t [WHERE expr]
EXPLAIN SELECT ...          SHOW TABLES          DESCRIBE t
```

Expressões aceitam `= != <> < <= > >=`, `AND OR NOT`, `IS [NOT] NULL`, `[NOT] LIKE` (com `%` e `_`, sem diferenciar maiúsculas), `[NOT] IN (...)`, `[NOT] BETWEEN a AND b`, aritmética `+ - * / %`, concatenação `||` e as funções `LOWER UPPER LENGTH ABS ROUND COALESCE`.

Planos de execução

Plano	Quando é escolhido	Custo
busca pela chave primária	WHERE tem `pk = literal`	Uma leitura pontual
busca pelo índice	WHERE tem `coluna_indexada = literal`	Uma varredura curta no índice e uma leitura por linha
varredura completa	Demais casos	Lê todas as linhas da tabela

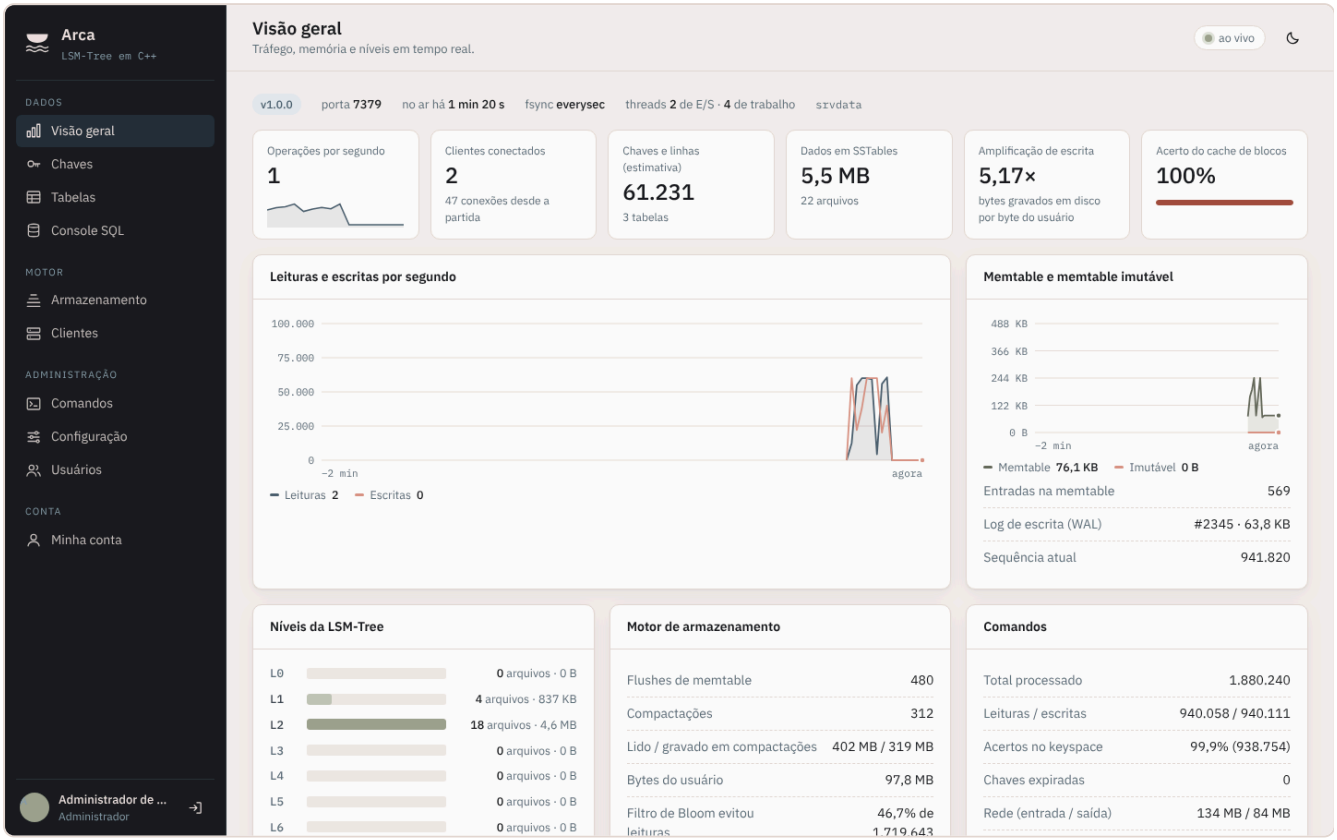
```
EXPLAIN SELECT * FROM pedidos WHERE status = 'pago'
plano: busca pelo índice idx_pedidos_status em pedidos
```

Regras de escrita

- Toda tabela precisa de chave primária. Colunas `AUTOINCREMENT` recebem o próximo número quando o valor é omitido ou NULL.
- Chave primária duplicada e NULL em coluna `NOT NULL` rejeitam a instrução inteira; nada é gravado.
- Cada instrução vira um único lote: linhas, entradas de índice e contadores mudam juntos.
- Escritas numa tabela são serializadas por uma trava de leitura e escrita da própria tabela; tabelas diferentes não se bloqueiam.

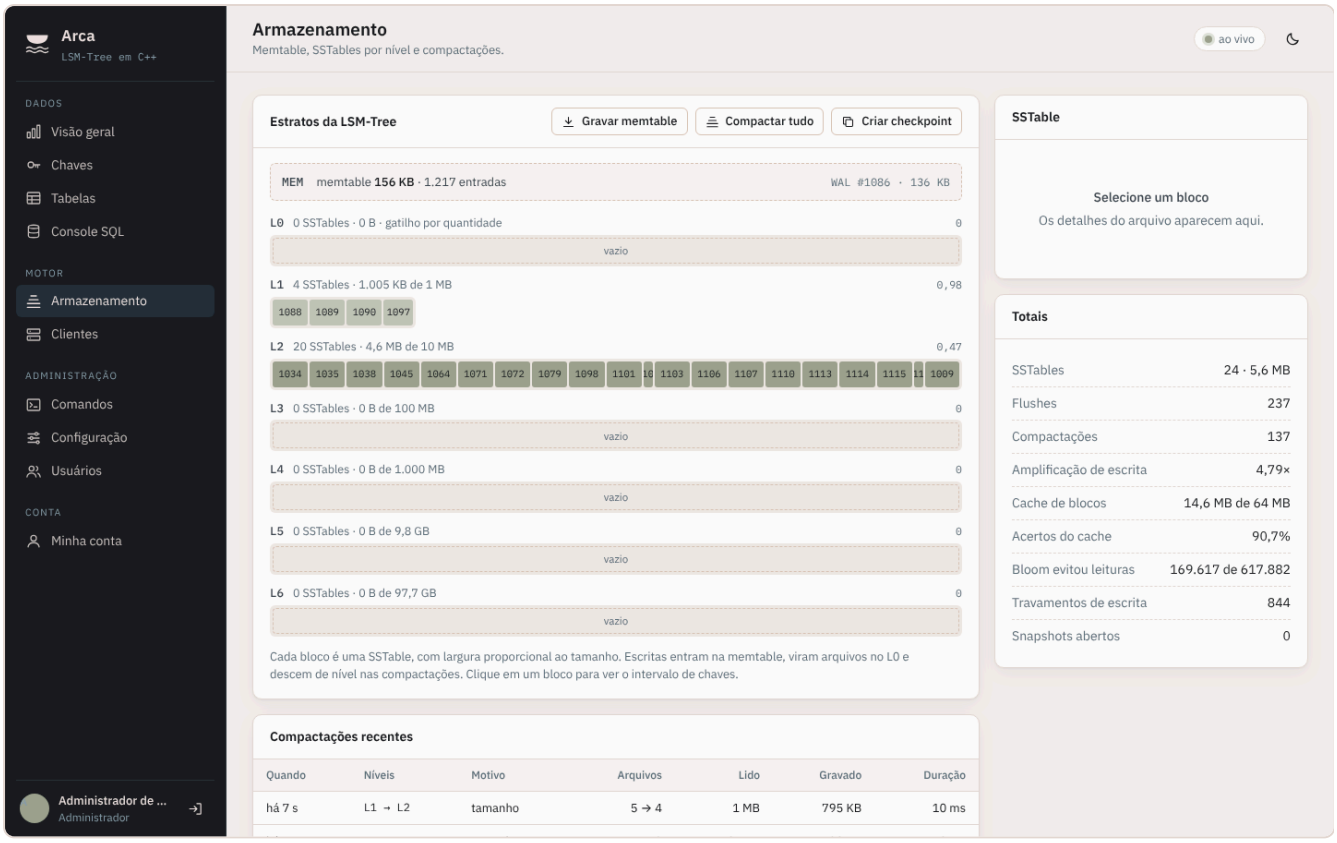
6 O painel

O painel mostra o estado do motor em tempo real (a cada segundo, por Server-Sent Events) e permite operar o banco sem linha de comando. Ele se adapta ao celular e tem tema claro e escuro.

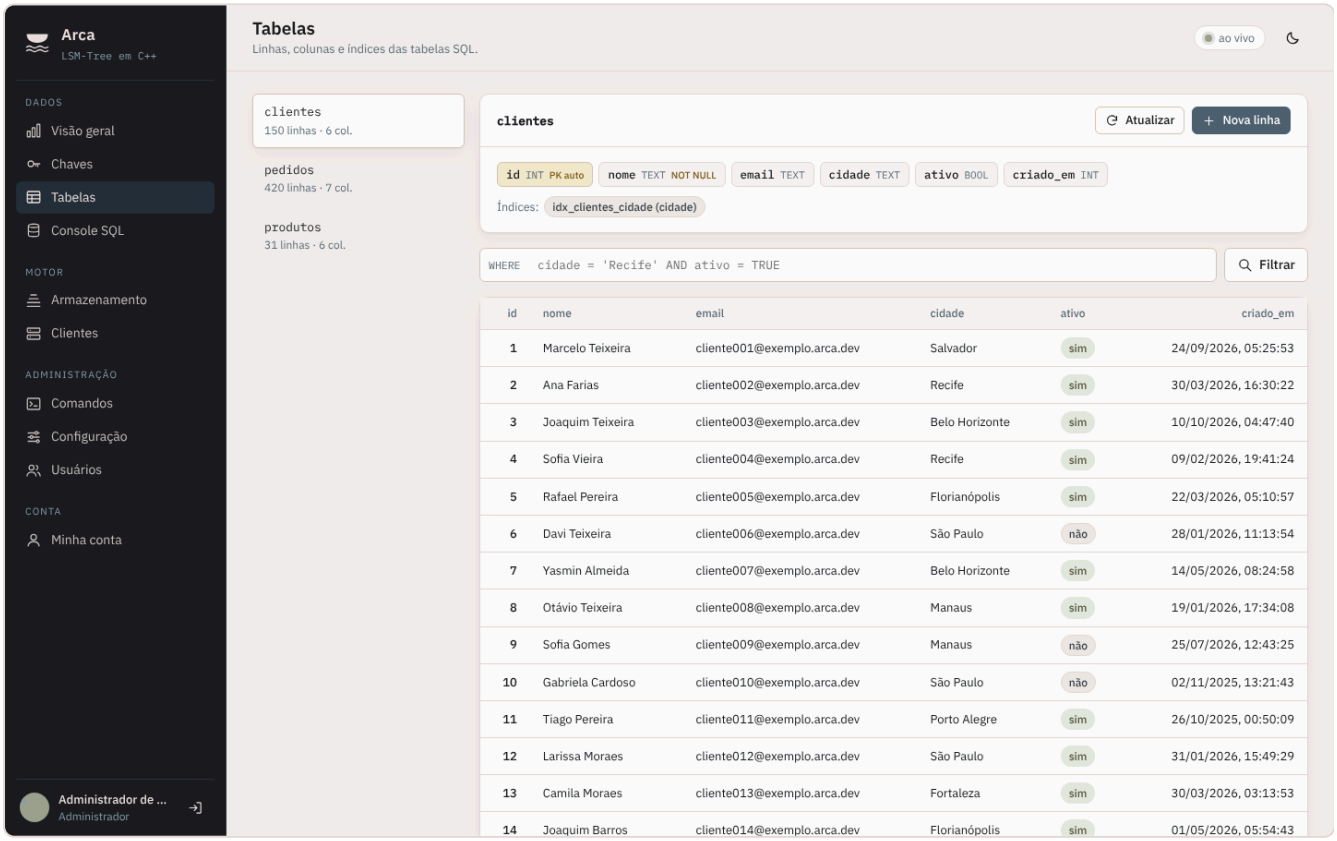


Visão geral: operações por segundo, memtable, níveis, amplificação de escrita e cache

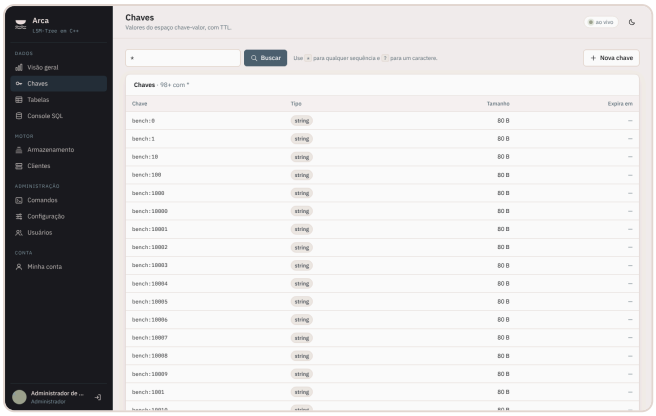
Tela	O que faz
Visão geral	Cartões com operações, clientes, chaves, SSTables, amplificação de escrita e acerto do cache; gráficos de leituras, escritas e memtable; pirâmide de níveis
Chaves	Busca por padrão (`sessao:*`), paginação por cursor, ver, editar, apagar e mudar o TTL
Tabelas	Lista de tabelas com colunas e índices; grade com ordenação, filtro em SQL, paginação e edição de linhas
Console SQL	Editor com exemplos, histórico, resultado em grade e o plano escolhido
Armazenamento	Memtable, WAL e cada SSTable por nível (largura proporcional ao tamanho), compactações recentes e botões para gravar a memtable, compactar e criar checkpoint
Clientes	Conexões abertas e comandos lentos (slowlog)
Comandos	Console direto do protocolo (só administrador)
Configuração	Parâmetros do servidor; os que podem mudar em execução são editáveis
Usuários	Contas, papéis, desbloqueio e a trilha de auditoria



Armazenamento: cada bloco é uma SSTable; a barra mostra o uso de cada nível



Tabelas: grade de linhas com filtro, ordenação e edição



Chaves com TTL



Painel no celular

7 API REST

O gateway expõe a API em `/api`. As respostas são JSON e as rotas, exceto login, saúde e o esquema OpenAPI, exigem o cabeçalho `Authorization: Bearer <token>`. Erros seguem sempre o formato `{"code": "...", "message": "...", "fields": {...}}`.

Rota	Papel mínimo	Descrição
POST /api/auth/login	—	Recebe `{email, password}` e devolve `{token, user, expires_at}`
POST /api/auth/logout · GET /api/auth/me · PUT /api/auth/me/password	Leitura	Sessão e conta
GET /api/overview · GET /api/storage	Leitura	Estatísticas do servidor, do motor e dos níveis
GET /api/events?token=	Leitura	SSE: evento `metrics` a cada segundo
GET /api/keys · GET /api/keys/value	Leitura	Varredura com cursor e leitura de uma chave
PUT /api/keys/value · DELETE /api/keys/value · POST /api/keys/expire	Operador	Gravar, apagar e mudar TTL
GET /api/tables · GET /api/tables/{nome}/rows	Leitura	Tabelas e linhas com filtro, ordem e paginação
POST · PUT · DELETE /api/tables/{nome}/rows	Operador	Inserir, alterar e apagar linhas pela chave primária
POST /api/sql	Leitura / Operador	Leitura só executa SELECT, EXPLAIN, SHOW e DESCRIBE
POST /api/storage/flush · compact · checkpoint	Operador	Operações do motor
GET /api/clients · GET /api/slowlog	Leitura	Conexões e comandos lentos
POST /api/command · GET/PUT /api/config	Administrador	Comando cru e configuração
GET · POST · PATCH · DELETE /api/users · GET /api/audit	Administrador	Usuários e auditoria
GET /api/health · GET /api/openapi.json	—	Saúde e esquema OpenAPI 3.1

```
TOKEN=$(curl -s -X POST localhost:8080/api/auth/login -H 'Content-Type: application/json' \
  -d '{"email":"operador@arca.dev","password":"Arca@2026"}' | jq -r .token)

curl -s -X POST localhost:8080/api/sql -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"query":"SELECT nome, preco FROM produtos ORDER BY preco DESC LIMIT 3"}'

curl -s -X PUT localhost:8080/api/keys/value -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"key":"config:tema","value":"escuro","ttl_s":600}'
```

8 Configuração

O servidor lê, nesta ordem: o arquivo passado como primeiro argumento (formato `nome valor`, uma linha por parâmetro), as variáveis de ambiente `ARCA_*` e os argumentos `--nome valor`. Tamanhos aceitam `kb`, `mb` e `gb`.

Parâmetro	Padrão	Variável	Em execução	Descrição
bind	127.0.0.1	ARCA_BIND	não	Endereço de escuta; 0.0.0.0 aceita conexões externas
port	7379	ARCA_PORT	não	Porta TCP
dir	dados	ARCA_DIR	não	Pasta do banco
requirepass	vazio	ARCA_PASSWORD	sim	Senha do AUTH
io-threads	2	ARCA_IO_THREADS	não	Laços de eventos de rede
worker-threads	4	ARCA_WORKER_THREADS	não	Threads para comandos pesados
appendfsync	everysec	ARCA_APPENDFSYNC	sim	always, everysec ou no
write-buffer-size	4mb	ARCA_WRITE_BUFFER_SIZE	não	Tamanho da memtable
max-file-size	2mb	—	não	Tamanho alvo das SSTables de compactação
level1-max-bytes	10mb	—	não	Limite do L1 (cada nível seguinte ×10)
block-cache-size	64mb	ARCA_BLOCK_CACHE_SIZE	não	Cache de blocos
bloom-bits-per-key	10	—	não	Precisão do filtro de Bloom
maxclients	10000	—	sim	Conexões simultâneas
timeout	0	—	sim	Segundos ociosos até desconectar (0 desliga)
proto-max-bulk-len	64mb	—	não	Maior argumento aceito
slowlog-log-slower-than	10000	—	sim	Microssegundos para entrar no slowlog (-1 desliga)
slowlog-max-len	128	—	sim	Entradas guardadas no slowlog
loglevel	info	—	sim	debug, info ou warning

Variável do gateway	Padrão	Descrição
ARCA_ADDR	127.0.0.1:7379	Endereço do servidor Arca
ARCA_PASSWORD	vazio	Senha do servidor

Variável do gateway	Padrão	Descrição
ARCA_HTTP_ADDR	:8080	Endereço HTTP do painel e da API
ARCA_WEB_DIR	../web/dist	Pasta do painel compilado
ARCA_SEED	true	Cria contas e dados de demonstração num banco sem usuários
ARCA_SESSION_HOURS	12	Validade das sessões do painel

9 Operação

Backup

`ARCA.CHECKPOINT nome` (ou o botão no painel) grava a memtable e cria em `checkpoints/nome` uma cópia consistente do banco usando links físicos, sem parar o servidor. A pasta é um banco completo: para restaurar, pare o servidor e aponte `dir` para ela, ou copie seu conteúdo.

```
arca-cli -a senha ARCA.CHECKPOINT antes-da-migracao
docker compose exec arca arca-cli ARCA.CHECKPOINT diario     # no Docker
docker run --rm -v lsm-tree-database-server_dados:/d -v "$PWD":/b alpine tar czf /b/backup.tgz
-C /d checkpoints
```

Monitoramento

- `INFO` traz memória, persistência, estatísticas e armazenamento no formato do Redis, então ferramentas de monitoramento de Redis conseguem ler.
- `ARCA.STATS` devolve tudo em JSON, inclusive cada SSTable e as últimas compactações.
- **Amplificação de escrita** = (bytes do usuário + flushes + compactação) ÷ bytes do usuário. Valores entre 2 e 10 são normais.
- **Travamentos de escrita** crescem quando o L0 acumula arquivos mais rápido do que a compactação consegue descer.

Ajuste de desempenho

Sintoma	Ajuste
Muitos travamentos de escrita	Aumente `write-buffer-size` (menos flushes) e `level1-max-bytes`
Leituras lentas em dados frios	Aumente `block-cache-size` ; `bloom-bits-per-key 14` reduz leituras inúteis
Latência alta com muitos clientes	Aumente `io-threads` até o número de núcleos
SQL pesado atrasando outros comandos	Aumente `worker-threads` e crie índices para os filtros mais usados
Precisa de durabilidade total	`appendfsync always` (cada escrita espera o fsync; o group commit reduz o custo)

Medindo

```
$ arca-bench -c 4 -n 60000 -P 32 -d 100
SET      176132 ops/s   p50     6.2 us   p99   210.7 us   erros 0
GET      327463 ops/s   p50     8.7 us   p99    65.3 us   erros 0
```

Valores medidos num contêiner Linux de 2 núcleos com `appendfsync everysec` ; o resultado depende do disco e da CPU.

Inspecionando os arquivos

```
arca-inspect check dados                          # confere todas as SSTables
arca-inspect sst dados/000042.sst --tudo          # propriedades e cada entrada
arca-inspect wal dados/000043.wal                 # lotes do log
arca-inspect manifest dados/MANIFEST-000044       # histórico de versões
```

10 Estendendo a Arca

Um comando novo

Os comandos ficam em `src/server/commands.cpp`. Escreva a função na struct `Commands` e registre o nome na tabela `CommandTable()` com a aridade (positiva é exata, negativa é mínima) e as flags.

```
static void Dobro(Server& s, Connection& c, Args& a, std::string* out) {
  KvRecord rec;
  if (!CheckRead(s, ReadKv(s, c, a[1], &rec), out)) return;
  resp::AppendBulk(out, rec.value + rec.value);
}

{"DOBRO", 2, kRead, Commands::Dobro},
```

Use `kWrite` para comandos que gravam, `kHeavy` para mandar ao pool de workers e `kNoMulti` para impedir o uso dentro de MULTI.

Uma função SQL

Inclua o nome em `ValidateFunctions` e o cálculo no ramo `kFunction` de `Eval`, em `src/sql/engine.cpp`.

Embutindo o motor

A biblioteca `arca_engine` não depende da rede nem do SQL. Inclua `arca/db.hpp`, ligue com `arca_engine` no CMake e use `DB::Open`, `Put`, `Get`, `Delete`, `Write` com lotes, `NewIterator`, `GetSnapshot`, `Flush`, `CompactAll`, `Checkpoint` e `GetStats`. O exemplo completo está em `examples/embedded.cpp`.

Gateway e painel

Rotas novas entram em `gateway/internal/api/`; o cliente RESP com pool em `gateway/internal/resp` já cuida de autenticação e reconexão. Telas novas entram em `web/src/pages/` e na lista de páginas de `web/src/App.tsx`.

11 Solução de problemas

Problema	Causa provável	Solução
`o diretório já está em uso por outro processo`	Outro arca-server abriu a mesma pasta	Pare o outro processo ou use outro `dir`
`bind na porta 7379` falhou	A porta está ocupada	Use `--port` ou pare o outro serviço
`NOAUTH autenticação necessária`	O servidor tem `requirepass`	Use `AUTH senha` ou `-a senha`
Painel mostra "Arca fora do ar"	O gateway não alcança o servidor	Confira `ARCA_ADDR`, a senha e `docker compose logs arca`
`N arquivo(s) SSTable ausentes` na abertura	Arquivo apagado ou cópia incompleta	Restaure de um checkpoint; o MANIFEST aponta exatamente o que falta
`descartando N bytes` no log da abertura	Queda no meio de uma gravação	Normal: só a cauda incompleta é descartada
Muitos "travamentos de escrita"	Compactação mais lenta que a entrada	Ver ajuste de desempenho no capítulo 9
`chave primária duplicada`	INSERT com chave que já existe	Use UPDATE ou outra chave
Conta do painel bloqueada	5 senhas erradas seguidas	Espere 5 minutos ou desbloqueie em Usuários
`iniciar.cmd` fecha sem subir	Docker Desktop não está rodando	Abra o Docker Desktop e rode de novo